

Cookie&Session 会话技术

今日内容介绍

- ◆ 案例一：记录用户的上次访问时间
- ◆ 案例二：一次性验证码

今日内容学习目标

- ◆ 可以响应给浏览器 Cookie 信息
- ◆ 可以接收浏览器携带的 cookie 信息
- ◆ 理解 cookie 的执行原理
- ◆ 可以对 session 作用域的数据进行操作：存放、获得、移除
- ◆ 理解 session 的执行原理

第1章 案例：记录用户的上次访问的时间：

1.1 需求：

当用户访问某些 Web 应用时，经常会显示出该用户上一次的访问时间。例如，QQ 登录成功后，会显示用户上次的登录时间。通过本任务，读者将学会如何使用 Cookie 技术实现显示用户上次的访问时间的功能。

1.2 相关知识点

1.2.1 会话的概述:

1.2.1.1 什么是会话

在日常生活中,从拨通电话到挂断电话之间的一连串的你问我答的过程就是一个会话。Web 应用中的会话过程类似于生活中的打电话过程,它指的是一个客户端(浏览器)与 Web 服务器之间连续发生的一系列请求和响应过程,例如,一个用户在某网站上的整个购物过程就是一个会话。

在打电话过程中,通话双方会有通话内容,同样,在客户端与服务器端交互的过程中,也会产生一些数据。例如,用户甲和乙分别登录了购物网站,甲购买了一个 Nokia 手机,乙购买了一个 Ipad,当这两个用户结账时,Web 服务器需要对用户甲和乙的信息分别进行保存。在前面章节讲解的对象中,HttpServletRequest 对象和 ServletContext 对象都可以对数据进行保存,但是这两个对象都不可行,具体原因如下:

(1) 客户端请求 Web 服务器时,针对每次 HTTP 请求,Web 服务器都会创建一个 HttpServletRequest 对象,该对象只能保存本次请求所传递的数据。由于购买和结账是两个不同的请求,因此,在发送结账请求时,之前购买请求中的数据将会丢失。

(2) 使用 ServletContext 对象保存数据时,由于同一个 Web 应用共享的是同一个 ServletContext 对象,因此,当用户在发送结账请求时,由于无法区分哪些商品是哪个用户所购买的,而会将该购物网站中所有用户购买的商品进行结算,这显然也是不可行的。

(3) 为了保存会话过程中产生的数据,在 Servlet 技术中,提供了两个用于保存会话数据的对象,分别是 Cookie 和 Session。关于 Cookie 和 Session 的相关知识,将在下面的小节进行详细讲解。



● Cookie 和浏览器缓存有什么区别?

共同点: 浏览器缓存可以缓存任意内容(上网浏览的所有内容)、cookie 只是服务器需要浏览器缓存数据(浏览器缓存中一部分)

1.2.1.2 什么是 Cookie

在现实生活中,当顾客在购物时,商城经常会赠送顾客一张会员卡,卡上记录用户的个人信息(姓名,手机号等)、消费额度和积分额度等。顾客一旦接受了会员卡,以后每次光临该商场时,都可以使用这张会员卡,商场也将根据会员卡上的消费记录计算会员的优惠额度和累加积分。在 Web 应用中, Cookie 的功能类似于这张会员卡,当用户通过浏览器访问 Web 服务器时,服务器会给客户端发

送一些信息，这些信息都保存在 Cookie 中。这样，当该浏览器再次访问服务器时，都会在请求头中将 Cookie 发送给服务器，方便服务器对浏览器做出正确的响应。

服务器向客户端发送 Cookie 时，会在 HTTP 响应头字段中增加 Set-Cookie 响应头字段。Set-Cookie 头字段中设置的 Cookie 遵循一定的语法格式，具体示例如下：

```
Set-Cookie: user=itcast; Path=/;
```

在上述示例中，user 表示 Cookie 的名称，itcast 表示 Cookie 的值，Path 表示 Cookie 的属性。需要注意的是，Cookie 必须以键值对的形式存在，其属性可以有多个，但这些属性之间必须用分号；和空格分隔。

了解了 Cookie 信息的发送方式后，接下来，通过一张图来描述 Cookie 在浏览器和服务器之间的传输过程，具体如图 5-1 所示。

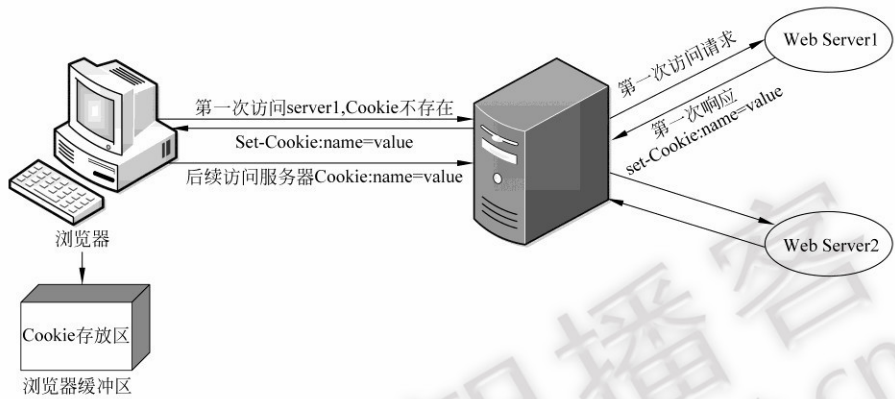


图 5-1 Cookie 在浏览器和服务器之间传输的过程

在图 5-1 中，描述了 Cookie 在浏览器和服务器之间的传输过程。当用户第一次访问服务器时，服务器会在响应消息中增加 Set-Cookie 头字段，将用户信息以 Cookie 的形式发送给浏览器。一旦用户浏览器接受了服务器发送的 Cookie 信息，就会将它保存在浏览器的缓冲区内，这样，当浏览器后续访问该服务器时，都会在请求消息中将用户信息以 Cookie 的形式发送给 Web 服务器，从而使服务器端分辨出当前请求是由哪个用户发出的。



1.2.1.3 Cookie 的基本使用

Cookie 将用户的信息保存到客户端浏览器的一个技术,当下次访问的时候,浏览器会自动携带 Cookie 的信息过来到服务器端。

● Cookie 的基本使用:

创建 cookie :	new Cookie(name,value) , javax.servlet.http.Cookie
将 cookie 发送给浏览器:	HttpServletResponse.addCookie(javax.servlet.http.Cookie)

接收浏览器携带的所有 cookie:	HttpServletRequest.getCookies()
--------------------	---------------------------------

1.3 案例分析

1.3.1 流程分析



1.3.2 步骤分析

第一步. 获得从客户端带过来的所有的 Cookie:

第二步. 从所有的 Cookie 中查找指定名称的 Cookie:

第三步. 判断是否是第一次访问:

- * 是第一次: 显示欢迎

- * 不是第一次: 显示欢迎 同时显示上次访问时间.

第四步. 记录当前的时间, 并且利用 Cookie 将时间回写到浏览器端.

1.4 代码实现

```
public class VisitServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        /**
         * 1. 获得从客户端带过来的所有的 Cookie:
         * 2. 从所有的 Cookie 中查找指定名称的 Cookie:
         * 3. 判断是否是第一次访问:
```

```

    *   * 是第一次: 显示欢迎
    *   * 不是第一次: 显示欢迎 同时显示上次访问时间.
    *   4. 记录当前的时间, 并且利用 Cookie 将时间回写到浏览器端.
    */
// 获得客户端的所有的 Cookie:
Cookie[] cookies = request.getCookies();
// 从 cookies 的数组中查找指定名称的 Cookie:
Cookie cookie = CookieUtils.findCookie(cookies, "visitTime");
// 判断是否是第一次访问:
response.setContentType("text/html;charset=UTF-8");
if(cookie == null){
    // 第一次访问
    response.getWriter().println("<h1>欢迎来到本网站!</h1>");
}else{
    // 不是第一次
    long time = Long.parseLong(cookie.getValue());
    Date date = new Date(time);
    response.getWriter().println("<h1>欢迎来到本网站!您的上次访问时间是:"+date.toLocaleString()+"</h1>");
}
// 记录当前的时间, 回写到 Cookie 中.
Cookie c = new Cookie("visitTime", ""+System.currentTimeMillis());
response.addCookie(c);
}

protected void doPost(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
    doGet(request, response);
}

}
```

1.5 总结:

1.5.1 Cookie 的分类

- 会话级别的 Cookie: **默认的**. 关闭了浏览器 Cookie 就销毁了.
- 持久级别的 Cookie: 需要设置有效时长的. 关闭浏览器也不会销毁的 Cookie.
 - `setMaxAge(int expiry);` 以秒为单位的时间, 超过了该时间后 Cookie 会自动销毁.
`setMaxAge(0)`, 手动删除持久性的 Cookie. (前提: `path` 和 `name` 必须一致)
 - `setPath(String uri);` 设置 Cookie 的有效路径.
 - ◆ 例如:
 - 1) `cookie.setPath("/day16/demo");`

表示 day16 项目下,【demo 目录】下所有的 servlet, 都可以访问当前 cookie。但“/day16”或“/day16/aaa”将不能访问。

2) cookie.setPath("/day16");

表示【day16 项目】下的所有 servlet 都可以访问当前 cookie

3) cookie.setPath("/");

表示【tomcat 下】的所有的 web 项目, 都可以访问当前 cookie

- cookie 唯一表示:

- 唯一标示: domain + path + name (类似 Java 中 包 + 类名)

- ◆ domain 域名, 不同的网站使用的是不同的域名, cookie 就不同。

- ◆ path 路径, 通过 cookie.setPath(...)设置的内容。

- ◆ name cookie 名称, 通过 new Cookie(name, ...) 确定的内容。

- 例如: 以下表示的是两个 cookie, 可以同时存在。

- ◆ /web/a/b/cookieName

- ◆ /web/a/cookieName

- 如果路径和名称一样, 两次 addCookie(), 后者将覆盖前者。

1.5.2 Cookie 的 API:

方法名	描述
getName()	获得 cookie 名称。
getValue()	获得 cookie 的值。
setMaxAge(int expiry)	设置 cookie 的有效时间。 ● 如果没有设置, cookie 只缓存浏览器缓存中, 浏览器关闭, cookie 删除。 ● 如果设置有效时间, 在时间范围内, cookie 被写入到浏览器端, 关闭浏览器下次访问仍可获得, 直到过期。
setPath(java.lang.String uri)	设置 cookie 允许被访问的路径。设置的路径, 以及子路径都被允许访问。 ● 例如: setPath("/web/a/b"); http://localhost:8080/web/a/b/oneServlet, 可访问(当前路径) http://localhost:8080/web/a/b/c/oneServlet, 可访问(子路径) http://localhost:8080/web/a/c/oneServlet, 不允许访问(无关路径) ● 常见设置: setPath("/") , 当前 tomcat 下的所有的 web 项目都可以访问

第2章 案例：一次性验证码的校验

2.1 需求:



- 验证码的作用:
 - 防止恶意提交表单

2.2 相关知识点:

2.2.1 Session 的概述

2.2.1.1 什么是 Session

当人们去医院就诊时，就诊病人需要办理医院的就诊卡，该卡上只有卡号，而没有其它信息。但病人每次去该医院就诊时，只要出示就诊卡，医务人员便可根据卡号查询到病人的就诊信息。Session 技术就好比医院发放给病人的就医卡和医院为每个病人保留病例档案的过程。当浏览器访问 Web 服务器时，Servlet 容器就会创建一个 Session 对象和 ID 属性，其中，Session 对象就相当于病历档案，ID 就相当于就诊卡号。当客户端后续访问服务器时，只要将标识号传递给服务器，服务器就能判断出该请求是哪个客户端发送的，从而选择与之对应的 Session 对象为其服务。

需要注意的是，由于客户端需要接收、记录和回送 Session 对象的 ID，因此，通常情况下，Session 是借助 Cookie 技术来传递 ID 属性的。

为了使读者更好的理解 Session，接下来，以网站购物为例，通过一张图来描述 Session 保存用户信息的原理，具体如图 5-5 所示。

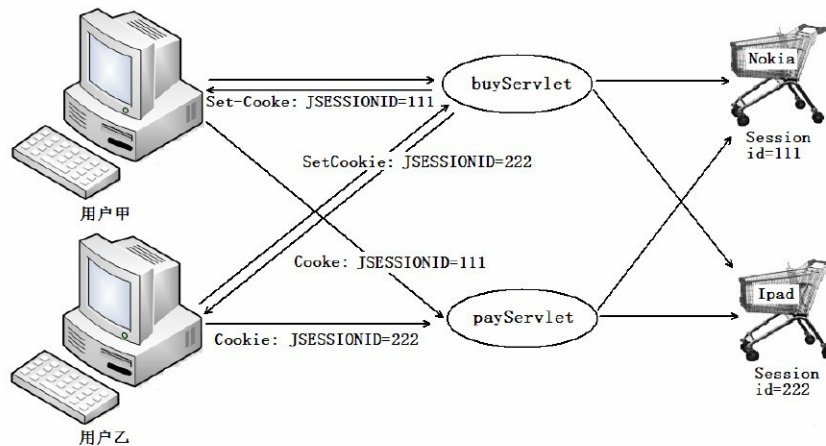
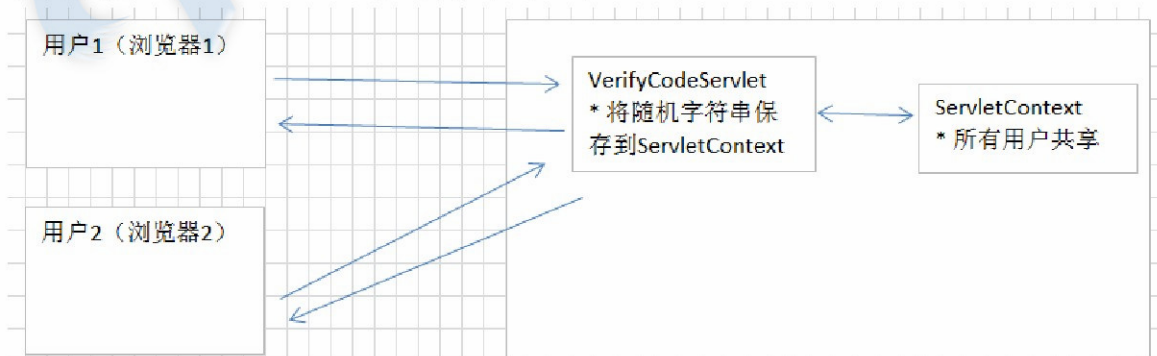


图1-1 Session 保存用户信息的过程

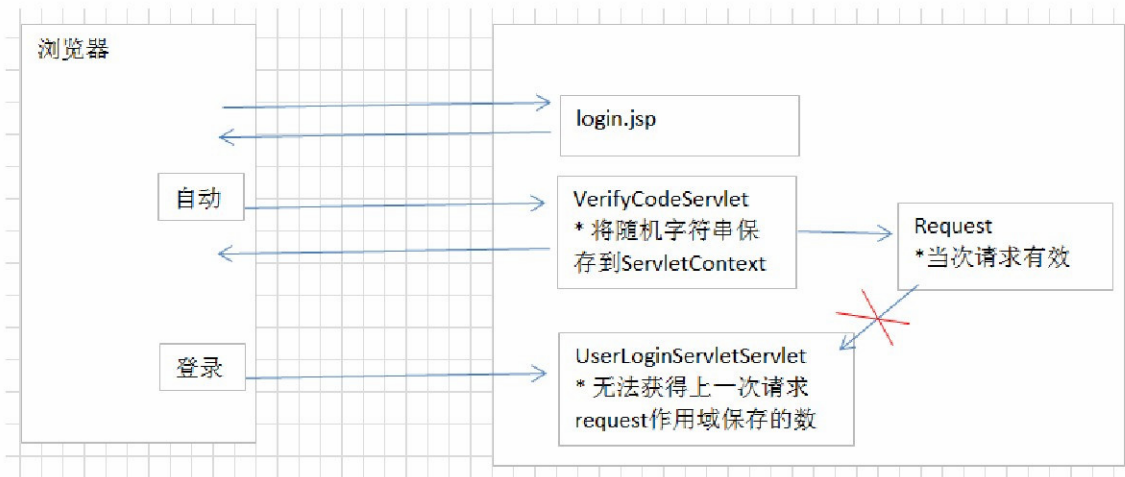
在图 5-5 中，用户甲和乙都调用 buyServlet 将商品添加到购物车，调用 payServlet 进行商品结算。由于甲和乙购买商品的过程类似，在此，以用户甲为例进行详细说明。当用户甲访问购物网站时，服务器为甲创建了一个 Session 对象（相当于购物车）。当甲将 Nokia 手机添加到购物车时，Nokia 手机的信息便存放到了 Session 对象中。同时，服务器将 Session 对象的 ID 属性以 Cookie (Set-Cookie: JSESSIONID=111) 的形式返回给甲的浏览器。当甲完成购物进行结账时，需要向服务器发送结账请求，这时，浏览器自动在请求消息头中将 Cookie (Cookie: JSESSIONID=111) 信息回送给服务器，服务器根据 ID 属性找到为用户甲所创建的 Session 对象，并将 Session 对象中所存放的 Nokia 手机信息取出进行结算。

2.2.1.2 为什么要使用会话

- 使用 servlet 生成验证码时，我们需要在服务器记录一份生成的随机字符，当用户提交填写的数据时，将用户输入的数据和服务器缓存的数据进行比对。
- 将随机字符串保存在 ServletContext 中或者 request 中是否可以？
 - 将数据存放到 ServletContext，多个用户共享一个验证码。



- 将数据存放到 request 作用域，多次请求不能共享数据。



- 我们发现将数据保存到 ServletContext 和 request 中是存在问题的, 那么就需要使用会话技术保存用户的私有信息.

2.2.1.3 有了 Cookie 为什么还有 Session?

- * Cookie 是有大小和个数的限制的.Session 存到服务器端的技术,没有大小和个数的限制.
- * Cookie 相对于 Session 来讲不安全.

2.2.1.4 如何使用 Session:

Session 是与每个请求消息紧密相关的, 为此, HttpServletRequest 定义了用于获取 Session 对象的 getSession()方法, 该方法有两种重载形式, 具体如下。

```
public HttpSession getSession(boolean create)
public HttpSession getSession()
```

上面重载的两个方法都用于返回与当前请求相关的 HttpSession 对象。不同的是, 第一个 getSession()方法根据传递的参数来判断是否创建新的 HttpSession 对象, 如果参数为 true, 则在相关的 HttpSession 对象不存在时创建并返回新的 HttpSession 对象, 否则不创建新的 HttpSession 对象, 而是返回 null。第二个 getSession()方法则相当于第一个方法参数为 true 时的情况, 在相关的 HttpSession 对象不存在时总是创建新的 HttpSession 对象。

要想使用 HttpSession 对象管理会话数据, 不仅需要获取到 HttpSession 对象, 还需要了解 HttpSession 对象的相关方法。HttpSession 接口中定义的操作会话数据的常用方法如表 5-2 所示。

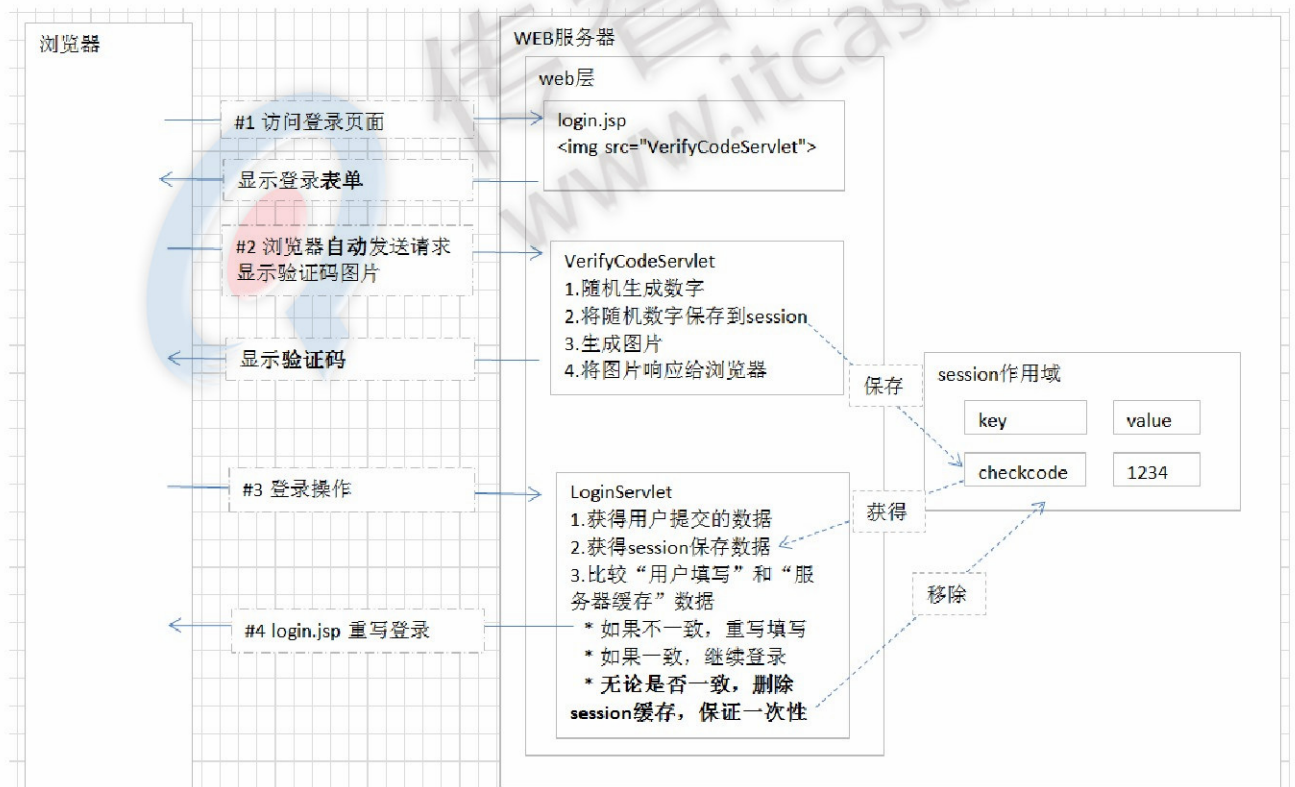
表5-1 HttpSession 接口中的常用方法

方法声明	功能描述
String getId()	用于返回与当前 HttpSession 对象关联的会话标识号
long getCreationTime()	返回 Session 创建的时间, 这个时间是创建 Session 的时间与 1970 年 1 月 1 日 00:00:00 之间时间差的毫秒表示形式

<code>long getLastAccessedTime()</code>	返回客户端最后一次发送与 Session 相关请求的时间，这个时间是发送请求的时间与 1970 年 1 月 1 日 00:00:00 之间时间差的毫秒表示形式
<code>void setMaxInactiveInterval(int interval)</code>	用于设置当前 HttpSession 对象可空闲的以秒为单位的最长时间，也就是修改当前会话的默认超时间隔
<code>boolean isNew()</code>	判断当前 HttpSession 对象是否是新创建的
<code>void invalidate()</code>	用于强制使 Session 对象无效
<code>ServletContext getServletContext()</code>	用于返回当前 HttpSession 对象所属于的 WEB 应用程序对象，即代表当前 WEB 应用程序的 ServletContext 对象
<code>void setAttribute(String name, Object value)</code>	用于将一个对象与一个名称关联后存储到当前的 HttpSession 对象中
<code>String getAttribute()</code>	用于从当前 HttpSession 对象中返回指定名称的属性对象
<code>void removeAttribute(String name)</code>	用于从当前 HttpSession 对象中删除指定名称的属性

表 5-2 列举了 HttpSession 接口中的常用方法，这些方法都是用来操作 HttpSession 对象的。

2.3 案例分析



2.4 代码实现:

- 将随机生成的字母或数字存入到 Session 中:

```
VerifyCodeServlet.java
44
45  /**#1 提供存放对象 */
46  StringBuilder builder = new StringBuilder();
47
48  //5 写随机字
49  for(int i = 0 ; i < 4 ; i++){
50      // 设置颜色--随机数
51      g.setColor(new Color(random.nextInt(255), random.nextInt(255), random.nextInt(255)));
52
53      // 获得随机字
54      int index = random.nextInt(data.length());
55      String str = data.substring(index, index + 1);
56      /**#2 存放随机字符 */
57      builder.append(str);
58
59      // 写入
60      g.drawString(str, width/6 * (i + 1), 20);
61
62  }
63  /**#3 将存放的4个字, 字符串 存放到session中 */
64  request.getSession().setAttribute("checkcode", builder.toString());
65
```

- 验证码的校验:

```
public class LoginServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
        // 进行验证码校验:
        String code1 = request.getParameter("checkcode");
        String code2 = (String)request.getSession().getAttribute("checkcode");
        request.getSession().removeAttribute("checkcode");
        if(!code1.equalsIgnoreCase(code2)){
            // 验证码有错误:
            request.setAttribute("msg", "验证码输入错误!");

            request.getRequestDispatcher("/demo4-checkcode/login.jsp").forward(request,
            response);

            return;
        }

        // 登录代码:

    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
        doGet(request, response);
    }
}
```



```
}
```

```
}
```

2.5 作用域总结:

2.5.1 Servlet 三个作用域总结:

- ServletContext: 针对一个 WEB 应用。一个 WEB 应用只有一个 ServletContext 对象, 使用该对象保存的数据在整个 WEB 应用中都有效。
 - 创建: 服务器启动的时候.
 - 销毁: 服务器关闭的时候或者项目移除的时候.
- HttpSession: 针对一次会话。使用该对象保存数据, 一次会话 (多次请求) 内数据有效。
 - 创建: 服务器第一次调用 getSession() 的时候, 服务器创建 session 的对象.
 - 销毁:
 1. 非正常关闭服务器 (正常关闭: Session 被序列化)
 2. Session 过期了, 默认时间是 30 分钟.
 3. 手动调用 session 的 invalidate 的方法.
- HttpServletRequest: 针对一次请求。使用该对象保存数据, 一次请求 (一个页面, 如果是请求转发多个页面) 内数据有效。
 - 创建: 客户端向服务器发送一次请求
 - 销毁: 服务器为这次请求作出响应之后, 销毁 request.
- 三个作用域对象操作的 API 相同
 - 存放数据: setAttribute(name, value)
 - 获得数据: getAttribute(name)
 - 删除数据: removeAttribute(name)

```
ServletRequest
  getAttribute(String) : Object
  getAttributeNames() : Enumeration<String>
  setAttribute(String, Object) : void
  removeAttribute(String) : void
```

```
HttpSession
  getAttribute(String) : Object
  getAttributeNames() : Enumeration<String>
  setAttribute(String, Object) : void
  removeAttribute(String) : void
```

```
ServletContext
  ● A getAttribute(String) : Object
  ● A getAttributeNames() : Enumeration<String>
  ● A setAttribute(String, Object) : void
  ● A removeAttribute(String) : void
```

第3章 总结

